

MÜŞTERİ HARCAMA DAVRANIŞLARI ÜRÜN KATEGORİSİ VE RİSK ANALİZİ TAHİMİNİ

Yapmış olduğum bu projede müşteri alışveriş verilerini kullanarak veri analiz işlemlerini gerçekleştirdim. Projemde veri setimi Kaggle'dan, analiz işlemlerini ise Hadoop platformundan yaptım. Veri setimde çeşitli analizler, ürün kategorisi tahmini ve iade riski analizi yapmak amacıyla makine öğrenmesi yöntemlerini uyguladım.

1. Projenin Amacı

Projemde müşteri alışveriş verilerini analiz ettim. Kategori tahminindeki amacım alışveriş bilgilerine bakarak kategori tahmini yapmak, risk analizinde ise müşterilerin aldığı ürünü iade etme riskini tahmin eden işlemleri uyguladık.

2. Kullanılan Teknolojiler

Projemde veri seti kaggle platformundan, analiz işlemleri için Hadoop ekosistemi kullandım, hadoop büyük veri setlerindeki işlemler için hızlı ve dağıtık çalışma imkanlarını sunar.

- Apache Spark (PySpark)
- Spark MLlib
- Lojistik Regresyon
- Karar Ağacı (Decision Tree)
- Pipeline, StringIndexer, VectorAssembler, StandardScaler

3. Veri Seti ve Ön İşleme

Projemin formatı CSV şeklindedir. Veri setimde müşterilerin, müşteri kimliği, yaş, cinsiyet, ürün kategorisi, ürün fiyatı, ürün adedi, ödeme yöntemi, alışveriş merkezi bilgileri yer almaktadır. Uyguladığım makine öğrenmesi algoritmalarının hatasız çalışması için öncesinde metinsel verileri sayısal hale getirdim. Cinsiyet, kategoriler, ödeme yöntemi ve alışveriş merkezi sütunlarını StringIndexer yöntemi ile sayısal değerlere dönüştürdüm ardından birleştirme ve ölçeklendirme işlem adımlarını uyguladım.

4. Modelleme Süreci

Projemde yaptığım analizleri iki farklı makine öğrenmesi algoritması kullanarak yaptım. Bunlar lojistik regresyon ve karar ağacı algoritmalarıdır. Veri setimi %80 eğitim ve %20 test olacak şekilde ikiye ayırarak eğitim verisi üzerinde eğitip, test verisi üzerinde tahminler yapacak şekilde ayarladım. Modellerin performansını test verisi üzerinde doğruluk (accuracy), F1-score, precision ve recall metriklerini kullanarak değerlendirme sonuçlarını çıktı olarak aldım ve elde ettiğim sonuçları model performanslarını karşılaştırıp değerlendirme sürecim için kullandım.

5. Model Performans Karşılaştırması

Model	Accuracy	F1-Score	Precision	Recall
Karar Ağacı	%94.57	0.9425	0.9580	0.9457
Lojistik Regresyon	%77.66	0.7102	0.6711	0.7766

6. Sonuçların Yorumlanması

Karar Ağacı modelinin %94.57 doğruluk oranı gerçek hayat senaryoları için daha güvenilir kabul edilmiştir. F1-score ve precision değerlerinin doğruluk oranına yakın olması, modelin dengeli bir performans sergilediğini göstermektedir.

7. Risk Analizi

Belirlediğimiz koşullara göre satış kaybı ve iade riski analizi yapılmıştır. Yüksek fiyat, yüksek ürün adedi ve belirli yaş grupları riskli olarak değerlendirilmiştir. Bu sayede stratejik hamleler ona göre uygulanıp en az şekilde zararlı çıkılacak ve müşteri memnuniyeti zedelenmeyecek şekilde hamleler uygulanacaktır. Karar ağacı algoritmasını kullanarak risk tahmini yaptık ve model doğruluğunu hesapladık.

8. Ek Analizler

Projemde ürün kategorisi ve risk analizine ek olarak müşteri davranışları üzerine analizler yaptım ve tekrar alışveriş yapan müşteri varımı tespit ettim. Müşteri sadakati, yaş gruplarının kategori tercihleri, cinsiyet bazlı alışveriş eğilimleri ve AVM bazlı kategori yoğunluklarını analiz edip çıktı olarak sonuçlarını yazdırdım.

9. Sonuç ve Öneriler

Elde edilen sonuçlar doğrultusunda, müşteri harcama miktarı ve ürün fiyatının kategori tahmininde en etkili değişkenler olduğu görülmüştür. Gelecek çalışmalarda daha büyük veri setleri ve farklı algoritmalar kullanılarak model gelişimi devam ettirilebilir.

EKLER

Ek-1 Veri Seti Tanımı

Veri seti: customer.csv

Toplam kayıt sayısı

Kullanılan değişkenler:

- customer_id
- gender
- age
- category
- quantity
- price
- payment_method
- shopping_mall

Ek-2 Parametreler

Lojistik Regresyon:

- maxIter = 150
- regParam = 0.001
- elasticNetParam = 0.5

Karar Ağacı:

- maxDepth = 6
- maxBins = 32

Risk Modeli Karar Ağacı:

- maxDepth = 10
- maxBins = 64

EK-3: Model Çıktıları ve Analiz Sonuçları

EK-3.1 Veri Seti Önizleme

Bu kısımda kullandığım müşteri veri setinin ilk 5 satırını gösterdim amacım veri setinin okunurluğunu görmektir. Veri seti; müşteri yaşı, cinsiyeti, alışveriş tutarı, ürün kategorisi, ödeme yöntemi ve AVM bilgilerini içermektedir.

customer_id	age	price	quantity	gender	payment_method	category	shopping_mall	invoice_date
C100004	61.0	1500.4	5.0	Male	Credit Card	Clothing	Metrocity	26/11/2021
C100005	34.0	1200.34	2.0	Male	Cash	Shoes	Kanyon	3/3/2023
C100006	44.0	107.52	3.0	Male	Credit Card	Toys	Cevahir AVM	1/12/2022
C100012	25.0	26.15	5.0	Male	Cash	Food & Beverage	Kanyon	15/08/2021
C100019	21.0	35.84	1.0	Female	Credit Card	Toys	Metrocity	25/07/2021

only showing top 5 rows

EK-3.2 Karar Ağacı Örnek Tahminleri

Karar Ağacı modelini kullanarak test verisi üzerinde yaptığım örnek tahminleri aşağıda gösterilmiştir. Tablo; müşterinin temel özellikleri ile modelin tahmin ettiği ürün kategorisini içermektedir.

```
KARAR AĞACI ÖRNEK TAHMİNLERİ:
+---+-----+-----+-----+-----+-----+
| age|gender| price|quantity| category|prediction|
+---+-----+-----+-----+-----+-----+
|44.0| Male|107.52| 3.0| Toys| 1.0|
|21.0|Female| 15.15| 1.0| Books| 6.0|
|60.0| Male|600.16| 2.0| Clothing| 0.0|
|62.0|Female|600.17| 1.0| Shoes| 4.0|
|64.0| Male| 45.45| 3.0| Books| 5.0|
|28.0|Female|1500.4| 5.0| Clothing| 0.0|
|45.0|Female| 15.69| 3.0|Food & Beverage| 2.0|
|63.0|Female|900.24| 3.0| Clothing| 0.0|
|54.0| Male|900.24| 3.0| Clothing| 0.0|
|33.0|Female| 179.2| 5.0| Toys| 3.0|
+---+-----+-----+-----+-----+-----+
only showing top 10 rows
#####
```

EK-3.3 Model Doğruluk Karşılaştırması

Lojistik Regresyon ve Karar Ağacı modellerinin doğruluk oranları karşılaştırılmıştır.

- Lojistik Regresyon Doğruluğu: %77.66
- Karar Ağacı Doğruluğu: %94.57

```
#####
### MÜŞTERİ DAVRANIŞI ANALİZ RAPORU ###
### Hedef: Ürün Kategorisi Tahmini ###
#####
Lojistik Regresyon Doğruluğu: % 77.66
Karar Ağacı Doğruluğu: % 94.57
```

EK-3.4 Detaylı Performans Metrikleri

Her iki model için F1 Score, Precision ve Recall değerleri hesaplanmıştır.

- **Karar Ağacı Modeli:**
 - F1 Score: 0.9425
 - Precision: 0.9580
 - Recall: 0.9457
- **Lojistik Regresyon Modeli:**
 - F1 Score: 0.7102
 - Precision: 0.6711
 - Recall: 0.7766

```
-----  
DECİSİON TREE DETAYLI METRİKLER (F1, PRECİSİON, RECALL):  
F1 Score:  0.9425  
Precision:  0.9580  
Recall:    0.9457  
-----  
LOJİSTİK REGRESYON DETAYLI METRİKLER (F1, PRECİSİON, RECALL):  
F1 Score:  0.7102  
Precision:  0.6711  
Recall:    0.7766  
-----
```

EK-3.5 Kategori – Sayısal Değer Eşlemesi

Model çıktılarının yorumlanabilmesinin kolaylığı açısından ürün kategorilerinin sayısal karşılıkları aşağıda verilmiştir.

```
-----  
Kategorilerin hangi sayısal değere karşılık geldiği:  
-----
```

```
0.0 = Clothing  
1.0 = Cosmetics  
2.0 = Food & Beverage  
3.0 = Toys  
4.0 = Shoes  
5.0 = Souvenir  
6.0 = Books  
7.0 = Technology
```

EK-3.6 Kategori – Tablo Açıklaması

Çıktı tablosunda sütunların anlaşılabilirliğini ve mantığını netleştirmek için açıklama kısmını aşağıda ekledim.

```
=====  
Tablo Açıklaması:  
=====
```

1. PRICE (Fiyat): Müşterinin harcadığı para.
2. CATEGORY (Gerçek): Harcanan para ile alınan ürünün kategorisi.
3. PREDICTION (Model): Modelin sayısal tahmini.

```
-----  
Category ile Prediction birbirini tutuyorsa model tahmini doğrudur.
```

EK-3.7 Kategori – Karmaşıklık Matrisi

Hangi kategoride ne kadar yanılma olduğunu görebilmek için matris aşağıda verilmiştir.

```
--- KARMAŞIKLIK MATRİSİ ---
```

category_prediction	0.0	1.0	2.0	3.0	4.0	5.0	6.0	7.0
Food & Beverage	0	0	145	0	0	0	0	0
Clothing	335	0	0	0	0	0	0	0
Souvenir	0	0	0	0	0	44	0	0
Books	0	0	0	0	0	9	16	0
Technology	0	0	0	0	0	0	0	50
Cosmetics	0	138	0	0	0	0	0	0
Shoes	0	0	0	0	114	0	0	0
Toys	0	43	0	64	0	0	0	0

EK-3.8 Kategori – Müşteri Analiz Raporu

Veri setimdeki işlemlerin anlık bağımsız fatura verileri olduğunu kanıtlamak için veri setimdeki müşteri ID bilgilerini kullanarak aynı müşteriden kaç tane olduğunu ayrıca oluşturduğum örnek veri seti ile model doğruluğumu aşağıda teyit ettim.

```
--- MÜŞTERİ ANALİZ RAPORU ---
```

SONUÇ: Her müşteri sadece 1 kez alışveriş yapmış. (Anlık Bağımsız Fatura Bilgisi)

```
--- TEST VERİSİ TEKRAR EDEN MÜŞTERİ KONTROLÜ ---
```

customer_id	count
TEST_01	2

=====

customer_id	category	quantity	price	prediction
TEST_01	Clothing	1.0	200.0	0.0
TEST_01	Food & Beverage	2.0	50.0	1.0

EK-3.9 Kategori – Cinsiyet Kategori Yönelimi

Hangi kategoride hangi cinsiyet yoğunluğunu aşağıda belirttim.

```
=====
CİNSİYET YÖNELİMİ: HANGİ KATEGORİDE KİM DAHA BASKIN?
=====
+-----+-----+-----+
| category|Female|Male|
+-----+-----+-----+
|Food & Beverage| 442| 286|
| Clothing| 1058| 664|
| Souvenir| 170| 99|
| Books| 148| 78|
| Technology| 126| 114|
| Cosmetics| 453| 308|
| Shoes| 310| 214|
| Toys| 320| 210|
+-----+-----+-----+
=====
```

EK-3.10 Kategori – Avm Bazlı Kategori Yoğunluğu

Hangi avm de hangi kategoriler ağırlıklı alınmış bunun bilgisini aşağıda belirttim.

```
=====
AVM BAZLI KATEGORİ YOĞUNLUĞU
=====
+-----+-----+-----+-----+-----+-----+-----+-----+
| shopping_mall|Books|Clothing|Cosmetics|Food & Beverage|Shoes|Souvenir|Technology|Toys|
+-----+-----+-----+-----+-----+-----+-----+-----+
| Kanyon| 56| 350| 152| 151| 88| 45| 40| 84|
| Metrocity| 26| 269| 133| 118| 82| 35| 36| 90|
| Mall of Istanbul| 47| 333| 162| 123| 111| 51| 58| 114|
| Zorlu Center| 8| 83| 36| 38| 34| 24| 14| 20|
| Cevahir AVM| 11| 87| 37| 30| 39| 15| 11| 30|
| Emaar Square Mall| 10| 98| 31| 31| 25| 17| 12| 35|
| Viaport Outlet| 11| 70| 33| 40| 25| 18| 11| 17|
| Istinye Park| 18| 164| 66| 77| 51| 26| 24| 61|
| Forum Istanbul| 17| 81| 40| 38| 21| 12| 10| 24|
| Metropol AVM| 22| 187| 71| 82| 48| 26| 24| 55|
+-----+-----+-----+-----+-----+-----+-----+-----+
=====
```

EK-3.11 Kategori – Kategori ve Ödeme Yöntemi Analizi

Hangi kategoride hangi ödeme yöntemi ağırlıklı kullanılmış bunun bilgisi.

STRATEJİK BAĞLANTI: KATEGORİ VE ÖDEME YÖNTEMİ ANALİZİ			
category	Cash	Credit Card	Debit Card
Food & Beverage	325	253	150
Clothing	773	603	346
Souvenir	133	78	58
Books	99	84	43
Technology	110	82	48
Cosmetics	340	256	165
Shoes	224	189	111
Toys	221	206	103

EK-3.12 Kategori – Yaş Gruplarının Kategori Tercihleri

Yaş gruplarını kategorilere ayırıp örneğin (20-30, 50-60) bu aralıklara göre yaş gruplarının hangi kategorilere yoğunlaştığını belirlemiş oldum.

YAŞ GRUPLARININ KATEGORİ TERCİHLERİ									
Age_Group	Books	Clothing	Cosmetics	Food & Beverage	Shoes	Souvenir	Technology	Toys	
10	15	76	28	27	17	14	9	16	
20	43	340	133	130	125	47	38	100	
30	45	321	164	137	92	51	58	113	
40	45	330	151	160	89	55	47	99	
50	35	317	148	131	90	48	45	101	
60	43	338	137	143	111	54	43	101	

EK-3.13 Kategori – Satış Kaybı ve İade Risk Tahmini

Karar Ağacı modeli kullanılarak test verisi üzerinde yapılan risk analizi aşağıda gösterilmiştir. Tablo; müşterinin belirlenen özellikler ile modelin tahmin ettiği risk türünü içermektedir.

```
=====
>>> SATIŞ KAYBI VE İADE RİSK TAHMİNİ
=====
```

age	shopping_mall	category	price	risk_label	prediction
25.0	Kanyon	Food & Beverage	26.15	0	0.0
50.0	Kanyon	Clothing	900.24	1	1.0
48.0	Cevahir AVM	Food & Beverage	5.23	0	0.0
31.0	Metrocity	Food & Beverage	26.15	0	0.0
48.0	Istinye Park	Clothing	1200.32	1	1.0
42.0	Zorlu Center	Shoes	1800.51	1	1.0
54.0	Viaport Outlet	Books	15.15	0	0.0
41.0	Viaport Outlet	Toys	143.36	0	0.0
26.0	Istinye Park	Toys	71.68	0	0.0
24.0	Metropol AVM	Shoes	600.17	0	0.0
66.0	Viaport Outlet	Clothing	900.24	1	0.0
25.0	Metrocity	Food & Beverage	5.23	0	0.0
21.0	Forum Istanbul	Shoes	1200.34	0	0.0
63.0	Emaar Square Mall	Clothing	1200.32	1	1.0
58.0	Metropol AVM	Clothing	1200.32	1	1.0
35.0	Metropol AVM	Technology	5250.0	0	0.0
36.0	Cevahir AVM	Food & Beverage	10.46	0	0.0
64.0	Istinye Park	Food & Beverage	5.23	0	0.0
26.0	Metrocity	Food & Beverage	15.69	0	0.0
40.0	Istinye Park	Cosmetics	121.98	0	0.0

```
only showing top 20 rows
RİSK ANALİZİ MODELİ DOĞRULUĞU: % 86.93
```

KOD AÇIKLAMALARI

1. Spark Ortamının Oluşturulması ve Gerekli Kütüphaneler

```
from pyspark.sql import SparkSession
from pyspark.ml import Pipeline
from pyspark.ml.feature import VectorAssembler, StandardScaler
from pyspark.ml.classification import LogisticRegression, DecisionTreeClassifier
from pyspark.ml.evaluation import BinaryClassificationEvaluator
from pyspark.ml.classification import LogisticRegressionModel
from pyspark.ml.feature import StringIndexer, VectorAssembler
from pyspark.ml.evaluation import RegressionEvaluator
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
from pyspark.sql.functions import col, when, count, avg, sum
```

Bu kod bloğunda proje boyunca kullanılacak olan PySpark kütüphanelerini projeye dahil ettim.

SparkSession, Apache Spark ortamını başlatmak için kullanılırken; MLlib kütüphaneleri makine öğrenmesi modelleri, veri ön işleme adımları ve performans ölçümleri için gereklidir. Ayrıca SQL fonksiyonları veri analizi ve ek iş zekâsı sorguları için projeye eklediğim kütüphanelerdir.

2. Spark Oturumunun Başlatılması ve Veri Setinin Yüklenmesi

```
spark = SparkSession.builder\
    .appName("LojistikRegresyon_Hadoop_Spark_Pipeline_KararAgaci")\
    .getOrCreate()

spark.sparkContext.setLogLevel("ERROR")

df = spark.read.csv(
    "/content/customer.csv",
    header=True,
    inferSchema=True
)
df.show(5)
```

Bu bölümde Spark oturumunu başlatıp proje için bir uygulama adı verdim.

CSV formatındaki müşteri verisi Spark DataFrame olarak içeri alınmıştır. inferSchema=True sayesinde veri tipleri otomatik olarak belirlenmiştir. İlk birkaç satırını göstererek veri yapısının kontrolünü sağladım.

3. Gereksiz veya Önceden Oluşturulmuş Sütunların Temizlenmesi

```
df = df.drop("category_index", "gender_index", "payment_index", "shopping_mall_index")
```

Bu adımda, daha önce oluşmuş olabilecek indeks sütunlarını temizledik. Böylece veri ön işleme süreci sıfırdan ve hatasız şekilde başlamıştır.

4. Kategorik Verilerin Sayısallaştırılması (StringIndexer)

```
indexer_gender = StringIndexer(inputCol="gender",  
outputCol="gender_index")  
  
indexer_category = StringIndexer(inputCol="category",  
outputCol="category_index")  
  
indexer_payment = StringIndexer(inputCol="payment_method",  
outputCol="payment_index")  
  
indexer_mall = StringIndexer(inputCol="shopping_mall",  
outputCol="shopping_index")
```

Bu kod bloğunda kategorik değişkenler (cinsiyet, ürün kategorisi, ödeme yöntemi ve AVM bilgisi) makine öğrenmesi algoritmalarının anlayabileceği sayısal formata dönüştürme işlemini gerçekleştirdim. StringIndexer, her kategoriye benzersiz bir sayısal etiket ile temsil eder.

5. Özelliklerin Birleştirilmesi ve Ölçeklendirilmesi

```
feature_cols=["age", "quantity", "price", "gender_index", "shopping_index", "payment_index"]  
  
assembler=VectorAssembler(  
    inputCols=feature_cols,  
    outputCol="features_raw"  
)  
  
scaler = StandardScaler(  
    inputCol="features_raw",  
    outputCol="features",  
    withMean=True,  
    withStd=True  
)
```

Bu aşamada modelde kullanılacak tüm özellikler tek bir vektör hâline getirilmiştir. Ardından StandardScaler kullanılarak özellikler normalize edilmiştir. Bu işlem özellikle Lojistik Regresyon gibi mesafeye duyarlı algoritmaların daha stabil çalışmasını sağlamaktadır.

6. Pipeline Oluşturulması ve Verinin Hazırlanması

```
pipeline= Pipeline(stages=[indexer_gender, indexer_category, indexer_payment, indexer_mall, assembler, scaler])  
train_df,test_df=df.randomSplit([0.8, 0.2], seed=42)  
  
prep_model=pipeline.fit(train_df)  
  
train_prepped=prep_model.transform(train_df)  
  
test_prepped=prep_model.transform(test_df)
```

Bu bölümde tüm veri ön işleme adımları tek bir Pipeline yapısı altında birleştirilmiştir. Veri seti %80 eğitim ve %20 test olarak ayrılmış, ardından pipeline sadece eğitim verisi üzerinde öğrenilmiştir. Böylece veri sızıntısının (data leakage) önüne geçilmiştir.

7. Lojistik Regresyon Modelinin Eğitilmesi ve Test Edilmesi

```
lr = LogisticRegression(  
    featuresCol="features",  
    labelCol="category_index", maxIter=150, elasticNetParam=0.5, regParam=0.001)  
lr_model=lr.fit(train_prepped)  
lr_predictions=lr_model.transform(test_prepped)
```

Bu kod bloğunda ürün kategorisi tahmini için Lojistik Regresyon modeli kurulmuştur. ElasticNet ve regularization parametreleri kullanılarak modelin aşırı öğrenmesi engellenmeye çalışılmıştır. Model, test verisi üzerinde tahminler üretmiştir.

8. Karar Ağacı Modelinin Eğitilmesi ve Test Edilmesi

```
dt= DecisionTreeClassifier(  
    featuresCol="features", maxDepth=6, maxBins=32,  
    labelCol="category_index")  
dt_model = dt.fit(train_prepped)  
dt_predictions=dt_model.transform(test_prepped)
```

Bu bölümde ikinci model olarak Karar Ağacı algoritması kullanılmıştır. Ağaç derinliği ve bölünme sayıları sınırlandırılarak daha gerçekçi bir model elde edilmesini amaçladım.

9. Model Performanslarının Değerlendirilmesi

```
acc_evaluator = MulticlassClassificationEvaluator(labelCol="category_index", metricName="accuracy")  
  
lr_acc=acc_evaluator.evaluate(lr_predictions)  
dt_acc=acc_evaluator.evaluate(dt_predictions)
```

Bu kod parçası ile her iki modelin doğruluk (accuracy) değerleri hesaplanmıştır. Bu sayede modellerin genel başarımı karşılaştırılabilir hâle getirilmiştir.

10. Ekrana Yazdırma

```
labels = prep_model.stages[1].labels

print("\n" + "#" * 60)
print("### MÜŞTERİ DAVRANIŞI ANALİZ RAPORU ###")
print("### Hedef: Ürün Kategorisi Tahmini ###")
print("#" * 60)
print(f"Lojistik Regresyon Doğruluğu: % {lr_acc * 100:.2f}")
print(f"Karar Ağacı Doğruluğu: % {dt_acc * 100:.2f}")
print("-" * 60)
```

Burada eğitilen makine öğrenmesi modellerinin genel performansını özetlemek amacıyla kullanılmıştır. Lojistik Regresyon ve Karar Ağacı modellerinin doğruluk oranları ekrana yazdırılarak, hangi modelin sınıflandırma başarısının daha yüksek olduğu net bir şekilde gösterilmiştir. Bu bölüm, raporun ana sonuç özetini oluşturmaktadır.

11. Detaylı Performans Metrikleri (F1, Precision, Recall)

```
evaluator = MulticlassClassificationEvaluator(labelCol="category_index", predictionCol="prediction")
f1_dt_val = evaluator.setMetricName("f1").evaluate(dt_predictions)
precision_dt_val = evaluator.setMetricName("weightedPrecision").evaluate(dt_predictions)
recall_dt_val = evaluator.setMetricName("weightedRecall").evaluate(dt_predictions)
```

Bu kod bloğunda Karar Ağacı modelinin performansı yalnızca doğruluk oranı ile sınırlı kalmayacak şekilde detaylandırılmıştır. F1-score, precision ve recall metrikleri hesaplanarak modelin sınıfları ne kadar dengeli ve doğru tahmin ettiği analiz edilmiştir. Özellikle çok sınıflı problemler için bu metrikler, modelin gerçek başarımını daha doğru vermektedir.

12. Lojistik Regresyon Modeli İçin Detaylı Metrik Analizi

```
evaluator = MulticlassClassificationEvaluator(labelCol="category_index", predictionCol="prediction")
f1_lr_val = evaluator.setMetricName("f1").evaluate(lr_predictions)
precision_lr_val = evaluator.setMetricName("weightedPrecision").evaluate(lr_predictions)
recall_lr_val = evaluator.setMetricName("weightedRecall").evaluate(lr_predictions)
```

Bu bölümde Lojistik Regresyon modeli için aynı performans metrikleri hesaplanmıştır. Her iki model için aynı değerlendirme ölçütlerinin kullanılması, modeller arasında daha adil ve tutarlı bir karşılaştırma yapmasını sağlamıştır.

13. Ekrana Yazdırma Kısımları

```
print("DECİSİON TREE DETAYLI METRİKLER (F1, PRECİSİON, RECALL):")
print(f"F1 Score:  {f1_dt_val:.4f}")
print(f"Precision:  {precision_dt_val:.4f}")
print(f"Recall:     {recall_dt_val:.4f}")
print("-" * 60)

print("LOJİSTİK REGRESYON DETAYLI METRİKLER (F1, PRECİSİON, RECALL):")
print(f"F1 Score:  {f1_lr_val:.4f}")
print(f"Precision:  {precision_lr_val:.4f}")
print(f"Recall:     {recall_lr_val:.4f}")
print("-" * 60)
```

Bu bölümde, modellerin performansı yalnızca doğruluk oranı ile değil; F1 Score, Precision ve Recall metrikleri ile detaylı olarak değerlendirilmiştir. Bu metrikler sayesinde modelin sınıfları ne kadar dengeli tahmin ettiği ve yanlış sınıflandırma oranları daha sağlıklı şekilde analiz edilmiştir.

14. Ekrana Yazdırma Kısımları

```
print("Kategorilerin hangi sayısal değere karşılık geldiği:")
print("-" * 60)
```

Bu başlık, model çıktılarının doğru yorumlanabilmesi için eklenmiştir.

15. Kategori Etiketlerinin Sayısal Karşılıklarının Gösterilmesi

```
labels = prep_model.stages[1].labels
for i, name in enumerate(labels):
    print(f"{float(i)} = {name}")
    print("-" * 60)
```

Bu kod parçası, model çıktılarında görülen sayısal etiketlerin hangi ürün kategorisine karşılık geldiğini açıklamak amacıyla kullanılmıştır. Bu sayede model tahminleri yorumlanabilir hâle getirilmiştir.

16. Ekrana Yazdırma Kısımları

```
print("KARAR AĞACI ÖRNEK TAHMİNLERİ:")
print("-" * 60)
```

Çıktıların karışmasını önlemek için başlıkları düzenli bir şekilde hazırladım.

17. Karar Ağacı Modeli Üzerinden Örnek Tahminlerin İncelenmesi

```
dt_predictions.select("age", "gender", "price", "quantity", "category", "prediction").show(10)
```

Bu bölümde Karar Ağacı modelinin ürettiği tahminler örnek veri kayıtları üzerinden gösterilmiştir. Gerçek kategori ile model tahmini karşılaştırılarak modelin fiyat, miktar ve müşteri bilgilerine göre doğru tahminler üretebildiğini görmüş olduk.

18. Ekran Yazdırma Kısımları

```
print("#" * 60 + "\n")  
print("\n" + "="*60)
```

Okunabilirliği artırmak ve rapor çıktısını düzenli hâle getirmek için kullandım.

19. Tahmin Tablosunun Açıklanması

```
print("Tablo Açıklaması:")
```

Bu açıklama kısmında fiyat, gerçek kategori ve model tahmini sütunlarının ne anlama geldiği kullanıcıya anlatılmıştır. Bu adım, raporu okuyan kişinin tabloyu teknik bilgiye sahip olmasa bile doğru yorumlayabilmesini sağlamaktadır.

20. Ekran Yazdırma Kısımları

```
print("="*60)  
print("1. PRICE (Fiyat): Müşterinin harcadığı para.")  
print("2. CATEGORY (Gerçek): Harcanan para ile alınan ürünün kategorisi.")  
print("3. PREDICTION (Model): Modelin sayısal tahmini.")  
print("-" * 60)  
print("Category ile Prediction birbirini tutuyorsa model tahmini doğrudur.")  
print("-" * 60)
```

Ekranında gösterilen tahmin tablosundaki sütunların ne anlama geldiği açıklanmıştır. Böylece tabloyu inceleyen kişinin sonuçları doğru anlaması sağlanmıştır.

21. Ekran Yazdırma Kısımları

```
print("--- KARMAŞIKLIK MATRİSİ ---")
```

Karar Ağacı modeline ait karmaşıklık matrisinin ekrana basılacağını belirtmek için kullanıldı.

22. Karmaşıklık Matrisi (Confusion Matrix) Oluşturulması

```
dt_predictions.stat.crosstab("category", "prediction").show()
```

Bu kod bloğu ile Karar Ağacı modelinin hangi kategorilerde doğru tahmin yaptığı, hangi kategorileri birbiriyle karıştırdığı analiz edilmiştir. Karmaşıklık matrisi, modelin güçlü ve zayıf yönlerini görsel olarak ortaya koymaktadır.

23. Tekrar Eden (Sadık) Müşterilerin Analizi

```
tekrar_edenler = df.groupby("customer_id").count().filter("count > 1")  
  
toplam_tekrar = tekrar_edenler.count()
```

Bu bölümde veri setinde birden fazla alışveriş yapan müşteriler tespit edilmiştir. Bu analiz, müşteri sadakati ve tekrar alışveriş davranışlarının incelenmesi açısından önemlidir.

24. Ekran Yazdırma Kısımları

```
print(f"\n--- MÜŞTERİ ANALİZ RAPORU ---")
```

Bu başlık, müşteri bazlı davranış analizine geçildiğini ifade eder.

25. Kontrol ve Ekran Yazdırma Kısımları

```
if toplam_tekrar > 0:  
    print(f"EFSANE HABER: Veri setinde {toplam_tekrar} tane tekrar alışveriş yapan sadık müşteri var!")  
    print("Örnek Sadık Müşteriler:")
```

Bu kod bloğu, veri setinde **birden fazla kez alışveriş yapan müşterilerin** olup olmadığını kontrol etmek için kullanılmıştır..

26. Örnek Tekrarlı Müşterilerin Gösterilmesi

```
tekrar_edenler.orderBy("count", ascending=False).show(5)  
else:  
    print("SONUÇ: Her müşteri sadece 1 kez alışveriş yapmış. (Anlık Bağımsız Fatura Bilgisi)")
```

Burada en fazla tekrar alışveriş yapan müşterileri listeleyerek modellerin tutarlılığını göstermek için kullandım.

27. Sonradan Eklenen İmport Kısmı

```
from pyspark.sql import Row
```

Spark DataFrame oluşturmak için manuel test verileri tanımlanmasını sağlar.

28. Manuel Test Verisi Oluşturulması ve Model Üzerinde Tahmin

```
test_data = [ Row(customer_id="TEST_01", gender="Male", age=25, category="Clothing", quantity=1.0, price=200.0, payment_method="Cash", shopping_mall="Kanyon"),  
              Row(customer_id="TEST_01", gender="Male", age=25, category="Food & Beverage", quantity=2.0, price=50.0, payment_method="Cash", shopping_mall="Kanyon")]
```

Bu bölümde modelin gerçek hayata benzer senaryolarda nasıl çalıştığını ve aynı müşteri verilerini gerçek veri setimde doğru hesaplandığını göstermek amacıyla teyit amaçlı kullandım.

29. Manuel Veri Test

```
sanal_df = spark.createDataFrame(test_data)  
  
print("\n--- TEST VERİSİ TEKRAR EDEN MÜŞTERİ KONTROLÜ ---")  
sanal_df.groupBy("customer_id").count().filter("count > 1").show()  
  
sanal_prepped = prep_model.transform(sanal_df)  
  
sanal_tahminler = lr_model.transform(sanal_prepped)  
  
print("\n" + "="*70)  
  
sanal_tahminler.select(  
    "customer_id",  
    "category",  
    "quantity",  
    "price",  
    "prediction").show()  
  
print("\n" + "="*70)  
print("CİNSİYET YÖNELİMİ: HANGİ KATEGORİDE KİM DAHA BASKIN?")  
print("="*70)
```

Modele ait çalışmayı göstermek amacıyla **elle oluşturmuş olduğum sanal müşteri verileri** kullanıldı.

Ayrıca test verileri üzerinde aynı müşteri kimliğine sahip birden fazla alışveriş kaydı oluşturularak tekrar eden müşteri analizinin çalıştığı gösterilmiştir.

Ekrana yazdırılan tabloda:

- Müşteri bilgileri,
 - Gerçek kategori,
 - Modelin tahmin ettiği kategori
birlikte gösterilerek modelin **fiyat ve miktar ilişkisini doğru kurup kurmadığı**
gözlemlenmiştir.
-

Cinsiyet Bazlı Kategori Eğilimi Başlığı

Bu kısım, bir sonraki adımda yapılacak **cinsiyetlere göre ürün kategorisi tercihi analizinin** başlangıcını belirtmek amacıyla ekrana başlık kısmı.

30. Cinsiyet Bazlı Kategori Tercih Analizi

```
yonelim_analizi = df.groupby("category").pivot("gender").count().fillna(0)
```

Bu analizde kadın ve erkek müşterilerin hangi ürün kategorilerinde daha yoğun alışveriş yaptığı incelenmiştir. Bu sonuçlar pazarlama ve hedefli kampanya stratejileri için önemli bilgiler sunması amaçlanmıştır.

31. Ekrana Çıktı Kısımları

```
yonelim_analizi.show()
```

```
mall_pivot.show()
```

```
payment_pivot.show()
```

```
age_analysis.show()
```

Bu satırlar, oluşturulan pivot tabloların ekrana basılmasını sağlar. Analiz sonuçlarının görsel ve tablo formatında incelenmesini sağlar.

32. AVM Bazlı Kategori Yoğunluğu Analizi

```
mall_pivot = df.groupby("shopping_mall").pivot("category").count().fillna(0)
```

Bu kod bloğu, farklı alışveriş merkezlerinde hangi ürün kategorilerinin daha yoğun satıldığını analiz etmektedir.

33. Kategori – Ödeme Yöntemi İlişkisi Analizi

```
payment_pivot = df.groupby("category").pivot("payment_method").count().fillna(0)
```

Bu analiz ile müşterilerin farklı ürün kategorilerinde hangi ödeme yöntemlerini tercih ettiği ortaya çıkarılmıştır. Bu sonuçlar, ödeme altyapısı ve kampanya planlaması açısından önemlidir.

34. Yaş Gruplarına Göre Kategori Tercihlerinin Analizi

```
df_with_age_bins = df.withColumn("Age_Group", (floor(df["age"] / 10) * 10))
```

Bu kod bloğunda müşteriler yaş gruplarına ayrılarak hangi yaş grubunun hangi kategorilere daha fazla yöneldiği analiz edilmiştir.

35. Satış Kaybı / İade Riski Etiketinin Oluşturulması

```
df_risk = train_prepped.withColumn("risk_label", when((col("price")>800) & (col("quantity") > 2) & (col("age") > 35), 1).otherwise(0))
```

Bu bölümde belirli kurallara göre riskli satışlar tanımlanmış ve veri setine yeni bir risk etiketi eklenmiştir risk analizi için gerek duydum.

36. Özellik Oluşturulması

```
feature_cols_risk = ["age", "gender_index", "shopping_index", "category_index"]
assembler_risk = VectorAssembler(inputCols=feature_cols_risk, outputCol="risk_features")

data_risk = assembler_risk.transform(df_risk)
train_r, test_r = data_risk.randomSplit([0.8, 0.2], seed=42)
```

Risk Özelliklerinin Oluşturulması:

Satış kaybı/iade riski tahmini için gerekli değişkenler birleştirilmiş ve model eğitimi için veri eğitim-test olarak ayrılmıştır.

37. Risk Tahmin Modelinin Eğitilmesi ve Değerlendirilmesi

```
dt_risk = DecisionTreeClassifier(featuresCol="risk_features", labelCol="risk_label", maxDepth=10, maxBins=64)
```

Son kısımda satış kaybı ve iade riskini tahmin etmek için ayrı bir Karar Ağacı modeli eğitmek ve doğruluk oranı hesapladım.

38. Risk Analiz Değerlendirme Kısım

```
model_risk = dt_risk.fit(train_r)
risk_predictions = model_risk.transform(test_r)

print("\n" + "-"*50)
print(">>> SATIŞ KAYBI VE İADE RİSK TAHMİNİ")
print("-"*50)
risk_predictions.select("age", "shopping_mall", "category", "price", "risk_label", "prediction").show(20)

evaluator_r = MulticlassClassificationEvaluator(labelCol="risk_label", predictionCol="prediction", metricName="accuracy")
print(f"RİSK ANALİZİ MODELİ DOĞRULUĞU: % {evaluator_r.evaluate(risk_predictions)*100:.2f}")
```

Risk Modeli Eğitimi ve Değerlendirilmesi:

Bu aşamada, daha önce belirlediğim risk etiketlerini kullanarak Karar Ağacı algoritması ile bir risk tahmin modeli eğittim. Eğitim verisi üzerinde oluşturduğum model, test verisinde uygulanarak müşterilerin satış kaybı veya iade riski taşıyıp taşımadığını tahmin etmek amacıyla hesaplandı.

Son kısımda ise çok sınıflı değerlendirme yöntemini kullanarak modelin doğruluk oranını hesapladım ve risk analizinde kullanılabilirliğini ölçtüm.